

Step 3: Implementation

BCS3210 Blockchains group project: Decentralized Digital Identity and Data Sharing Platform

What we did in this step

We implemented the design from Step 2: four Solidity contracts in a Hardhat 3 project (same setup as the labs: viem toolbox, Solidity 0.8.28, optimizer on), a Hardhat Ignition module that deploys and connects them, a demo script, and the off-chain part in Python (hashing, the patient's vault and an end-to-end demo). Everything runs on the local Hardhat network.

1. What was built

File	What it contains
contracts/AccessToken.sol	ERC-20 token "ACT" written by hand like in Tutorial 3, with a <code>minter</code> that is set to the <code>ConsentManager</code>
contracts/DigitalIdentity.sol	Patient registration (salted email hash, storage reference, SHA-256 of both photos), document update, requester approval by the admin
contracts/ConsentManager.sol	<code>grantConsent</code> (scope + 1-365 days, reward once per requester), <code>revokeConsent</code> , <code>isConsentValid</code> , <code>getConsent</code>
contracts/DataSharing.sol	<code>requestAccess</code> that never reverts on a missing consent, append-only audit log with reason codes, <code>pause/unpause</code>
contracts/interfaces/*.sol	Small interfaces used for the calls between contracts (as in Lab 4)
ignition/modules/Platform.ts	Deploys the four contracts in order and calls <code>setMinter</code>
scripts/demo.ts	Walks through the whole flow with viem, including a time jump for expiry
offchain/hash_tool.py	SHA-256 of files and the salted email hash (same result as <code>keccak256(abi.encodePacked(salt, email))</code>)
offchain/make_sample_data.py	Creates a fake patient with generated ID images (no real personal data)
offchain/vault.py	The patient's vault: checks the on-chain request and the signature before it releases files
offchain/demo.py	End-to-end demo with web3.py against <code>npx hardhat node</code>

2. Changes compared to our first version

We had a first version of the contracts from an earlier session. While implementing we found some problems and changed the design:

First version	Problem	Now
<code>getUserRecord()</code> returned the document link to anyone	The link was readable without consent, so the consent check protected nothing	Files are only handed out by the off-chain vault after a granted request on-chain
<code>requestAccess</code> returned the link and hashes	Return values of a transaction are not delivered to the caller	Returns true/false and emits an event; the vault checks that event
OpenZeppelin <code>Ownable</code> and <code>ERC20</code>	Fine, but we wanted to write it like in the course	<code>Own</code> owner + <code>onlyOwner</code> modifier and <code>own</code> <code>ERC-20</code>

First version	Problem	Now
5 contracts, separate logger that stored a string per entry	Extra contract call and expensive strings	4 contracts, log inside DataSharing with enum reasons
Email hash without salt	Can be reversed by trying common emails	Salted hash, salt kept by the patient
No check that the patient exists	Anyone could create log entries for random addresses	requestAccess requires a registered patient; unapproved requesters are logged as denied

3. How to run it

```
cd digital-identity-platform
npm install
npx hardhat compile
npx hardhat run scripts/demo.ts          # full demo on the in-process network

# off-chain demo against a real local node
npx hardhat node                        # terminal 1
npm run deploy:local                    # terminal 2
python -m venv .venv && .venv\Scripts\activate
pip install -r offchain/requirements.txt
python offchain/make_sample_data.py
python offchain/demo.py
```

4. Results

We checked that the contracts compile without warnings, that the Ignition module deploys on the in-process network and on `npx hardhat node`, and that both demos run from a fresh node. Output of `scripts/demo.ts` (shortened):

```
Step 4: Doctor requests access without consent...
  DENIED (reason NoConsent), gas used: 141712
Step 5: Patient grants consent (Both, 30 days)...
  Consent valid: true
  Patient ACT balance: 10
Step 6: Doctor requests access with consent...
  GRANTED (scope Both), gas used: 107430
Step 7: Stranger (not approved) requests access...
  DENIED (reason NotApproved), gas used: 92551
Step 8: Patient revokes consent...
  DENIED (reason Revoked), gas used: 107532
Step 9: Patient grants 7 days again, then 8 days pass...
  ACT balance (no second reward): 10
  DENIED (reason Expired), gas used: 107543
```

Output of `offchain/demo.py` (shortened):

```
4. Doctor requestAccess -> GRANTED
   Vault released: ['id_front.png', 'id_back.png']
   id_front.png hash matches on-chain
   id_back.png hash matches on-chain
5. Reusing the same request -> request already used
6. Stranger uses doctor's request -> signature does not match the requester
7. Front photo changed after download:
   id_front.png HASH MISMATCH, file was changed
8. Consent revoked, doctor requestAccess -> DENIED
   Vault -> access was not granted on-chain
9. Audit log of the patient (on-chain):
   #0 GRANTED doctor scope=2
   #1 GRANTED doctor scope=2
   #2 DENIED doctor reason=Revoked
```

The first request costs more gas (141712) than later ones (about 107500) because the log length and totalRequests go from zero to non-zero, and writing a storage slot from zero costs 20000 gas instead of 5000. Unit tests and proper gas measurements are part of Step 4.